INTERNSHIP PROJECT REPORT

CUSTOMER SERVICE CHATBOT DATA SCIENCE DASHBOARD

LEARN TO BUILD A REAL-TIME GEN AI CUSTOMER SERVICE BOT

Sentiment & Risk Analysis • Automated Ticketing • Knowledge Base • RAG • OCR • Multilingual Sessions

Submitted to

ElevanceSkills Technology Private Limited

Submitted by

SONALI SANJAY JADHAV

Role: Data Science Intern

Department: B.E. Artificial Intelligence and Data Science

College name: PES Modern College of Engineering, Shivajinagar, Pune

Internship Duration: 13 September 2026 – 13 October 2026

Academic Year: 2026–2027

Internship Organization: ElevanceSkills
Internship Mode: Remote
Project Type: Data Science

DECLARATION

I, Sonali Sanjay Jadhav, hereby declare that this report titled “AI-powered customer service chatbot” record of the work carried out by me during my Data Science Internship with ElevanceSkills Technology Private Limited.

The project focuses on a Python-based customer service chatbot and six additional internship task modules involving sentiment and risk analysis, automated support ticket management, knowledge-base monitoring, retrieval-augmented generation (RAG), OCR-based evidence processing, and multilingual conversation/session management.

I have prepared this report to describe the project scope, implementation approach, tools, learning outcomes, and supporting evidence. The source code and project documentation are maintained in the GitHub repository listed in this report. Any results or implementation details should be interpreted according to the code, configuration, datasets, and tests actually included in that repository.

Name: Sonali Sanjay Jadhav

Role: Data Science Intern

Organization: ElevanceSkills Technology Private Limited

Signature: 

Date: 13 October 2026

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to ElevanceSkills Technology Private Limited for providing me with the opportunity to complete a project-based Data Science Internship. The internship gave me an opportunity to connect programming and AI concepts with a practical customer-support use case.

I am thankful for the task requirements and learning resources that encouraged me to explore natural-language processing, sentiment analysis, retrieval-based question answering, document processing, ticket workflows, and multilingual conversation handling. Working through the six task modules also helped me practise breaking large requirements into smaller components and integrating them into a project.

I would also like to thank PES Modern College of Engineering, Shivajinagar, Pune, and the faculty members of the Artificial Intelligence and Data Science department for their academic support. Finally, I am grateful to everyone who supported me directly or indirectly during the internship.

Sonali Sanjay Jadhav

EXECUTIVE SUMMARY

This internship project is an AI-powered customer service chatbot designed to support common customer-service workflows. In addition to natural-language interaction, the project documentation describes modules for sentiment and risk analysis, structured support-ticket creation, knowledge-base maintenance, retrieval-augmented generation (RAG), OCR-based evidence extraction, and multilingual session management.

The project is implemented primarily in Python, with Streamlit used for the web interface. Depending on the enabled module and configuration, the technology stack includes large language models, NLP and sentiment-analysis logic, LangChain integration, vector similarity search using FAISS, OCR tools, Git/GitHub, and Python test scripts.

The six internship tasks are organised as independent functional areas: (1) multilingual sentiment and risk analysis, (2) automated support-ticket management, (3) knowledge-base monitoring and maintenance, (4) RAG-based question answering, (5) OCR-based evidence processing, and (6) multilingual conversation and session management. Together, these modules aim to make support interactions more organised, context-aware, and easier to escalate when needed.

This report describes the project background, objectives, data and knowledge sources, architecture, task-level requirements, implementation approach, testing considerations, challenges, outcomes, and future improvements. Specific quantitative performance claims are not included unless they can be verified from the project's executed tests or recorded outputs.

TABLE OF CONTENTS

Section	Page
1. Introduction	6
2. Project Background	6
3. Learning Objectives	6
4. Tools and Technologies Used	7
5. Data and Knowledge-Base Description	7
6. Project Methodology and Workflow	8
7. Activities and Internship Tasks	9
7.1 Task 1 – Multilingual Sentiment and Risk Analyzer	9
7.2 Task 2 – Automated Support Ticket Management	10
7.3 Task 3 – Knowledge-Base Monitoring and Maintenance	11
7.4 Task 4 – Retrieval-Augmented Generation (RAG)	12
7.5 Task 5 – OCR-Based Evidence Processing	13
7.6 Task 6 – Multilingual Conversation and Session Management	14
8. Testing and Validation Approach	15
9. Challenges and Solutions	15
10. Feedback and Evidence	16
11. Outcomes and Impact	20
12. Conclusion	21
13. References	21
14. Appendix	22

1. INTRODUCTION

Customer-support teams receive a wide variety of messages, from simple product questions to complaints, urgent issues, order problems, and requests that require human intervention. Handling these messages consistently can be difficult when the information is incomplete, spread across documents, or written in different languages.

An AI-powered customer service chatbot can assist by interpreting a user's message, finding relevant information, collecting missing details, and organising unresolved issues for follow-up. The goal is not to remove the need for human support agents, but to make routine handling more consistent and help important cases reach the appropriate team.

During the internship period from 13 September 2026 to 13 October 2026, the project work was organised around an existing customer-service chatbot training project and six additional task modules. The repository includes project folders for the source code, dataset, task modules, tests, and documentation.

1.1 Project Title

AI-Powered Customer Service Chatbot with Sentiment Analysis, Ticket Automation, RAG, OCR, and Multilingual Session Management.

1.2 Aim

To develop and organise a Python-based customer-support application that can analyse customer messages, assist with ticket creation, retrieve grounded answers from knowledge sources, process supported evidence files, and manage multilingual conversations.

2. PROJECT BACKGROUND

2.1 Background

Customer service is a suitable application area for NLP and language-model technologies because customer messages often contain informal wording, incomplete details, emotion, and references to earlier interactions. A support assistant must do more than generate fluent text: it should use relevant evidence, recognise uncertainty, protect session boundaries, and escalate cases that need human attention..

2.2 Proposed Approach

The project separates these concerns into six task modules. Each module handles a defined responsibility and can be tested against its own requirements. The Streamlit interface provides an accessible way to interact with configured chatbot functionality, while GitHub stores the source code and documentation.

2.3 Expected Users

- Customers seeking answers or help with an issue.
- Support agents who review escalated or unresolved cases.
- Knowledge-base maintainers responsible for document quality.
- Project developers who test, maintain, and extend the chatbot.

3. LEARNING OBJECTIVES

- Understand how NLP and language-model components can be used in customer-support workflows.
- Explore sentiment, urgency, and risk signals in customer messages.
- Represent unresolved issues as structured tickets with priority and routing information.
- Understand document validation, duplicate detection, versioning, and knowledge-base quality checks.

- Learn the retrieval-augmented generation pipeline and the importance of grounding answers in source material.
- Understand OCR as a method for extracting text and selected fields from supported evidence files.
- Manage multilingual dialogue, conversation context, and session isolation.
- Use Python project structure, dependency management, GitHub documentation, and testing practices.
- Improve debugging, requirement analysis, technical documentation, and problem-solving skills.

4. TOOLS AND TECHNOLOGIES USED

Technology	Purpose in the project
Python	Main programming language for application logic and task modules.
Streamlit	Browser-based interface for interacting with the configured chatbot and task workflows.
NLP / sentiment analysis	Interpret message tone and identify sentiment, frustration, urgency, or possible risk.
Large Language Models (LLMs)	Natural-language understanding and response generation where configured.
Retrieval-Augmented Generation (RAG)	Retrieve relevant document passages and use them as context for answers.
LangChain	Integration layer for LLM and retrieval components where configured.
FAISS	Vector similarity search for finding semantically relevant text chunks where configured.
OCR tools	Text extraction from supported images or documents.
Pandas / data utilities	Tabular data handling where used by the dataset or task implementation.
Git and GitHub	Version control, repository hosting, and project documentation.
Pytest / Python test scripts	Automated or repeatable checks where included in the repository.

5. DATA AND KNOWLEDGE-BASE DESCRIPTION

5.1 Project Dataset

The repository retains the original FAQ dataset at dataset/dataset.csv. It supports the original customer-service question-answering functionality. The README notes that Windows-1252 encoding may be required when loading this CSV. No record count is stated because it should be verified directly from the dataset.

5.2 Knowledge-Base Sources

The knowledge-base and RAG workflow is designed to work with documents such as frequently asked questions, policies, troubleshooting guides, and product-support information. These documents are used as retrieval sources so that answers can be based on relevant content rather than relying only on model-generated text.

5.3 Data Preparation and Document Quality

- Validate supported file types and document readability before processing.
- Identify duplicate or changed documents where the pipeline supports this operation.
- Track document versions and provide a rollback path where implemented.
- Use metadata such as product, region, document version, and effective dates where available.
- Handle missing, conflicting, or insufficient evidence explicitly instead of inventing values.

6. PROJECT METHODOLOGY AND WORKFLOW

6.1 Overall Workflow

Start → Customer Enters Query → Process User Input → Analyze Sentiment and Urgency → Retrieve Relevant Information from Knowledge Base → Generate AI Response → Check Whether the Issue Is Resolved → If Yes, Display Response → If No, Create Support Ticket or Escalate to Human Support → Display Response → End.

6.2 Methodology

1. Requirement analysis: break each internship task into input, processing, output, and edge-case requirements.
2. Module design: keep sentiment analysis, ticketing, knowledge-base processing, RAG, OCR, and session management logically separated.
3. Data and document preparation: validate supported inputs and prepare content for retrieval or extraction.
4. Processing: apply configured NLP, retrieval, extraction, routing, and response-generation logic.
5. Safety and reliability checks: handle low confidence, missing fields, insufficient evidence, and cases requiring escalation.
6. Testing: run module-level and integration checks against representative normal, boundary, and failure inputs.
7. Documentation and submission: maintain source code, dependencies, README instructions, and the final report in the GitHub repository.

6.3 RAG Pipeline

A typical RAG workflow splits source documents into chunks, creates embeddings for the chunks, stores them in a vector index, retrieves relevant chunks for a user question, and passes the retrieved context to the language model. The response can then cite or refer to supporting sources when the application has source metadata available. The precise embedding provider, vector store, chunking settings, and model are determined by the repository configuration.

6.4 Repository Structure

The following project structure is documented in the submitted GitHub repository README.

Folder / file	Purpose
dataset/	Dataset associated with the original project.
src/	Core chatbot source code.
task1/	Multilingual sentiment and risk analysis.
task2/	Automated support-ticket management.
task3/	Knowledge-base monitoring and maintenance.
task4/	RAG and document retrieval.
task5/	OCR and evidence processing.
task6/	Multilingual conversation and session management.
tests/	Test files and repeatable checks, where present.
requirements.txt	Python dependencies.
README.md	Project overview, setup, and usage documentation.

7. ACTIVITIES AND INTERNSHIP TASKS

7.1 TASK 1 — MULTILINGUAL SENTIMENT AND RISK ANALYZER

Objective

To analyse customer messages and relevant conversation context to identify sentiment, frustration, urgency, and possible risk, and to support suitable response tone or escalation.

Key Requirements

- Classify sentiment and risk using the configured analysis approach.
- Provide confidence information where the implementation supports it.
- Consider message context instead of relying only on isolated words.
- Adapt the response tone to the detected customer situation.
- Escalate urgent or high-risk issues according to configured rules.
- Handle after-hours or unresolved cases using the configured workflow.

Implementation Approach

The task receives a customer message and, where available, conversation history. The analysis component evaluates sentiment and risk signals and returns structured information that can guide the next step. Escalation should be driven by explicit rules and/or model outputs with appropriate safeguards. Ambiguous or low-confidence cases should not be treated as certain classifications.

Result

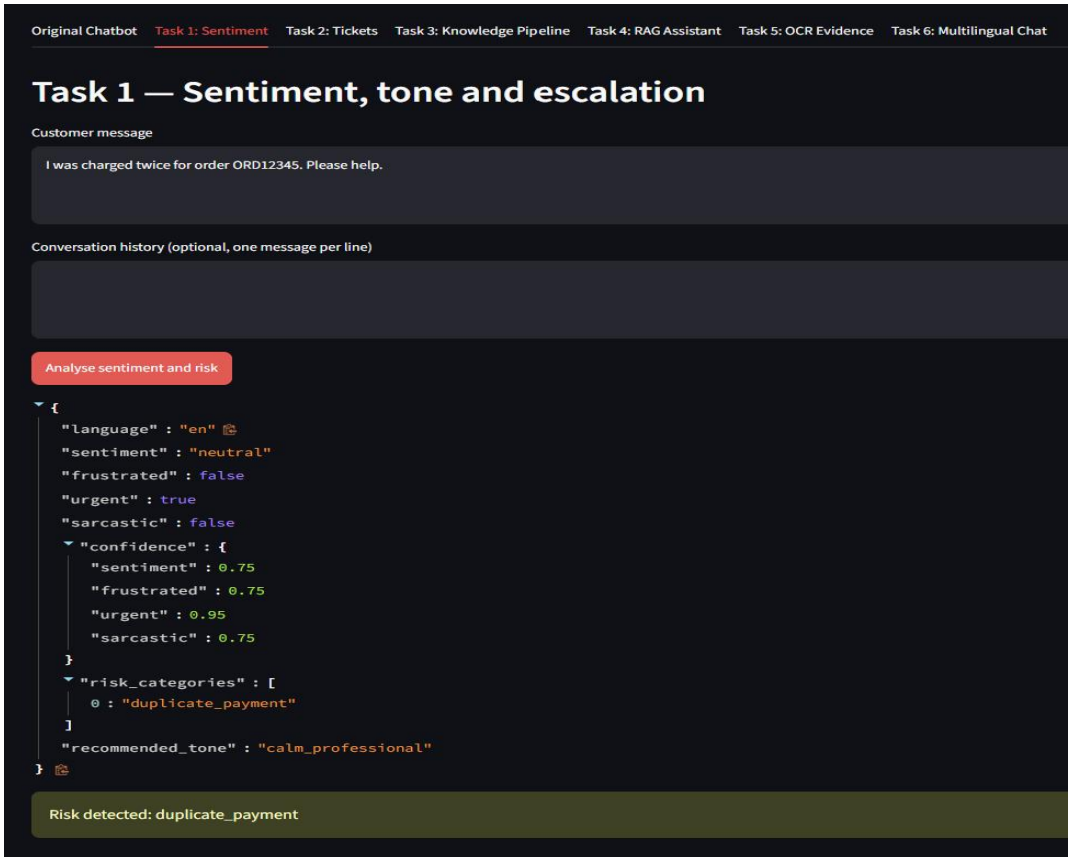


Figure 1: Task 1

7.2 TASK 2 — AUTOMATED SUPPORT TICKET MANAGEMENT

Objective

To convert unresolved customer conversations into structured support tickets and help route them to an appropriate team.

Key Requirements

- Extract customer, order, product, issue, and contact details where available.
- Identify missing mandatory fields and request clarification when needed.
- Assign priority using configured severity, impact, waiting-time, and SLA rules.
- Route tickets using available team skills, availability, workload, and business-hour rules.
- Detect duplicate or related tickets where supported.

Implementation Approach

The workflow collects relevant information from the conversation and maps it into a consistent ticket structure. Before ticket creation or routing, required fields should be validated. Priority and routing logic should be traceable to configuration or rules so that the outcome can be reviewed by support staff.

Result

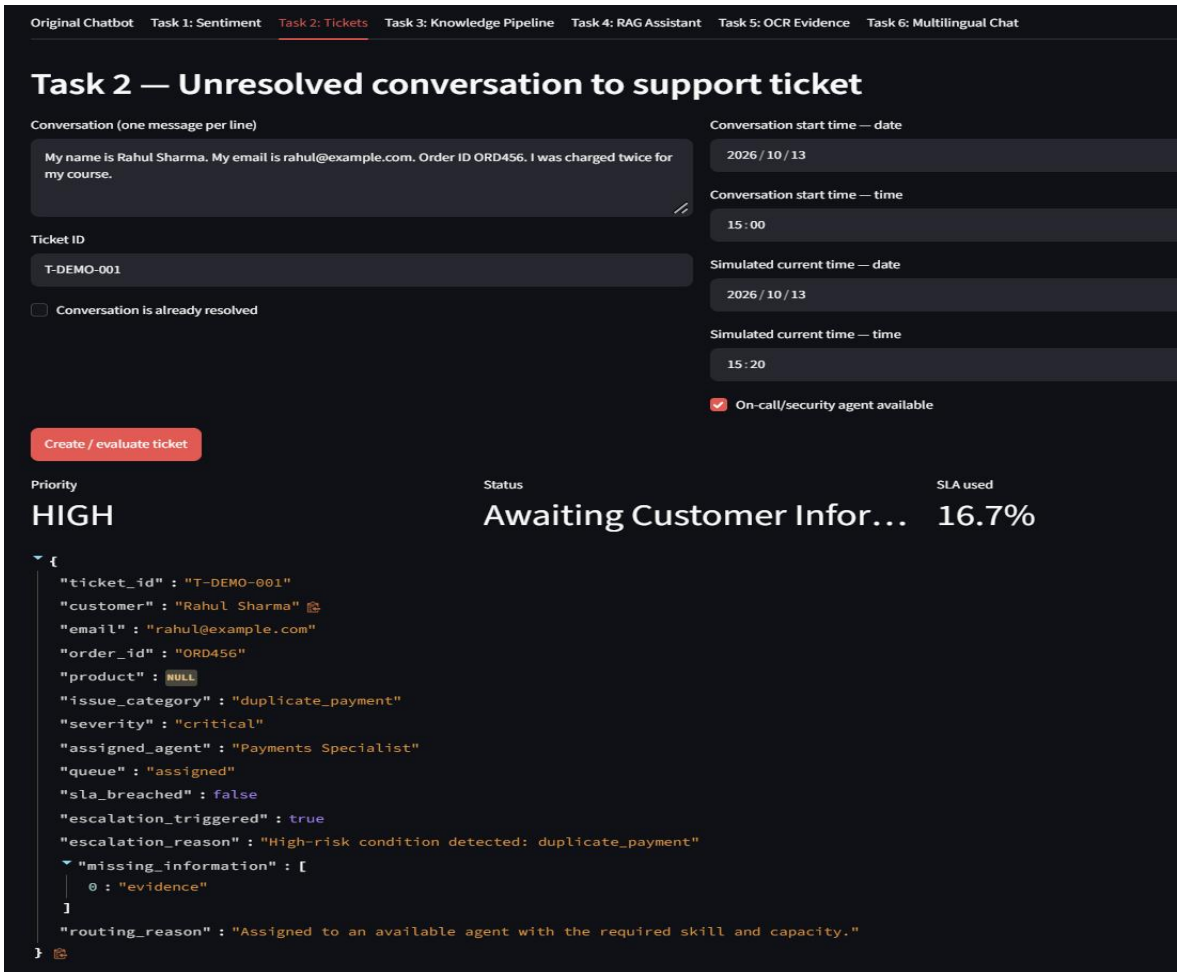


Figure 2: Task 2

7.3 TASK 3 — KNOWLEDGE-BASE MONITORING AND MAINTENANCE PIPELINE

Objective

To improve the quality and reliability of documents used by the chatbot by identifying changes, duplicates, invalid inputs, and quality issues.

Key Requirements

- Detect new, modified, duplicate, and invalid documents where supported.
- Manage document versions and provide rollback workflows where implemented.
- Check document quality and whether generated answers are grounded in retrieved content.
- Support scheduled processing, retries, and recovery from processing errors where configured.
- Monitor pipeline health and apply access-control and prompt-injection protections.

Implementation Approach

The pipeline validates documents before they are added to the retrieval corpus. A robust process tracks document identity and version, records processing status, and handles errors without silently treating failed ingestion as successful. Untrusted instructions inside documents should be treated as content rather than as commands to the system.

Result

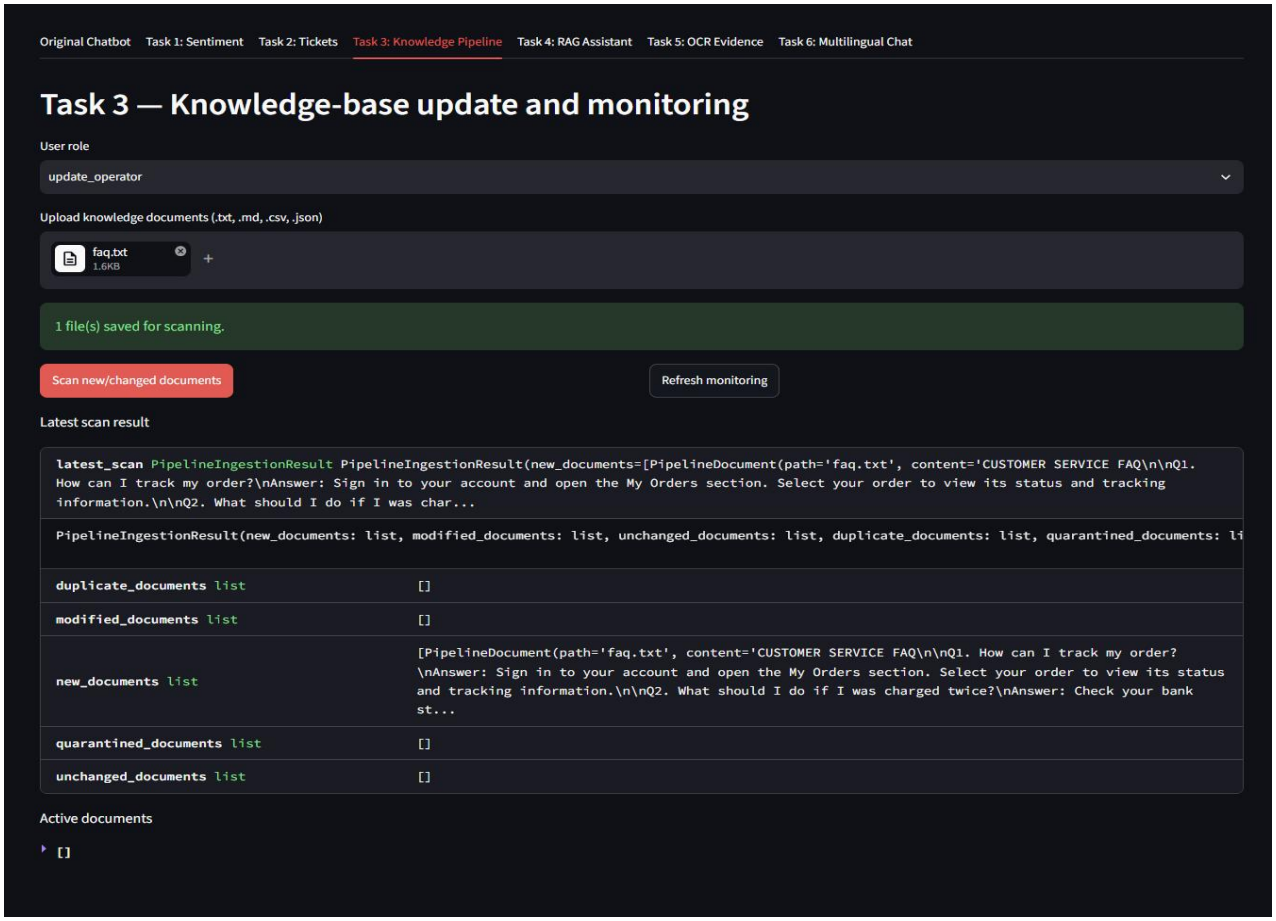


Figure 3: Task 3

7.4 TASK 4 — RETRIEVAL-AUGMENTED GENERATION (RAG)

Objective

To answer customer questions using relevant retrieved knowledge-base passages and to reduce unsupported responses.

Key Requirements

- Retrieve information from FAQs, policies, and troubleshooting documents.
- Use metadata such as version, product, region, and effective date when available.
- Select current or historical documents according to the question and metadata.
- Provide source citations when the source information is available.
- Ask for clarification when a question is ambiguous.
- Handle insufficient evidence and apply access restrictions to retrieved content.

Implementation Approach

The RAG pipeline converts source documents into searchable representations, retrieves relevant passages for a query, and supplies those passages as context to the response generator. A grounded answer should stay within the evidence retrieved. If no useful source is found, the assistant should state that the available knowledge is insufficient or direct the case to a human agent.

Result

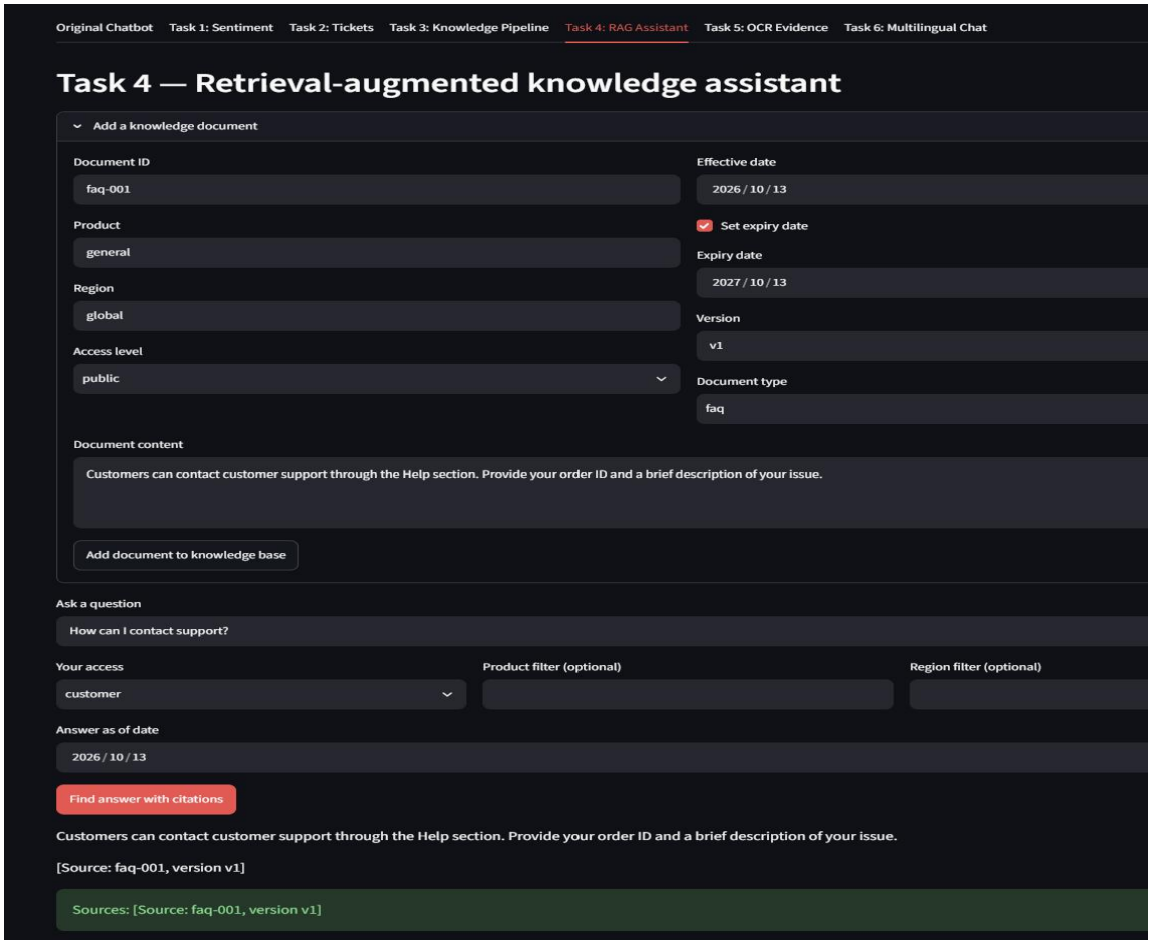


Figure 4: Task 4

7.5 TASK 5 — OCR-BASED EVIDENCE PROCESSING

Objective

To extract useful information from supported customer evidence such as screenshots, invoices, PDFs, and product-related images.

Key Requirements

- Extract fields such as order IDs, dates, amounts, product information, and error codes where visible.
- Handle missing, conflicting, or low-quality extracted information.
- Avoid inventing values that cannot be read from the evidence.
- Validate files and supported formats before processing.
- Apply sensitive-data masking and retention rules where configured.
- Support background processing when required by the application design.

Implementation Approach

The OCR workflow validates an uploaded file, extracts text, and then identifies relevant fields from the extracted content. Extracted values should be treated as candidates until validated. Low-quality images, unusual layouts, and ambiguous characters can produce errors, so the interface should allow clarification or manual correction.

Result

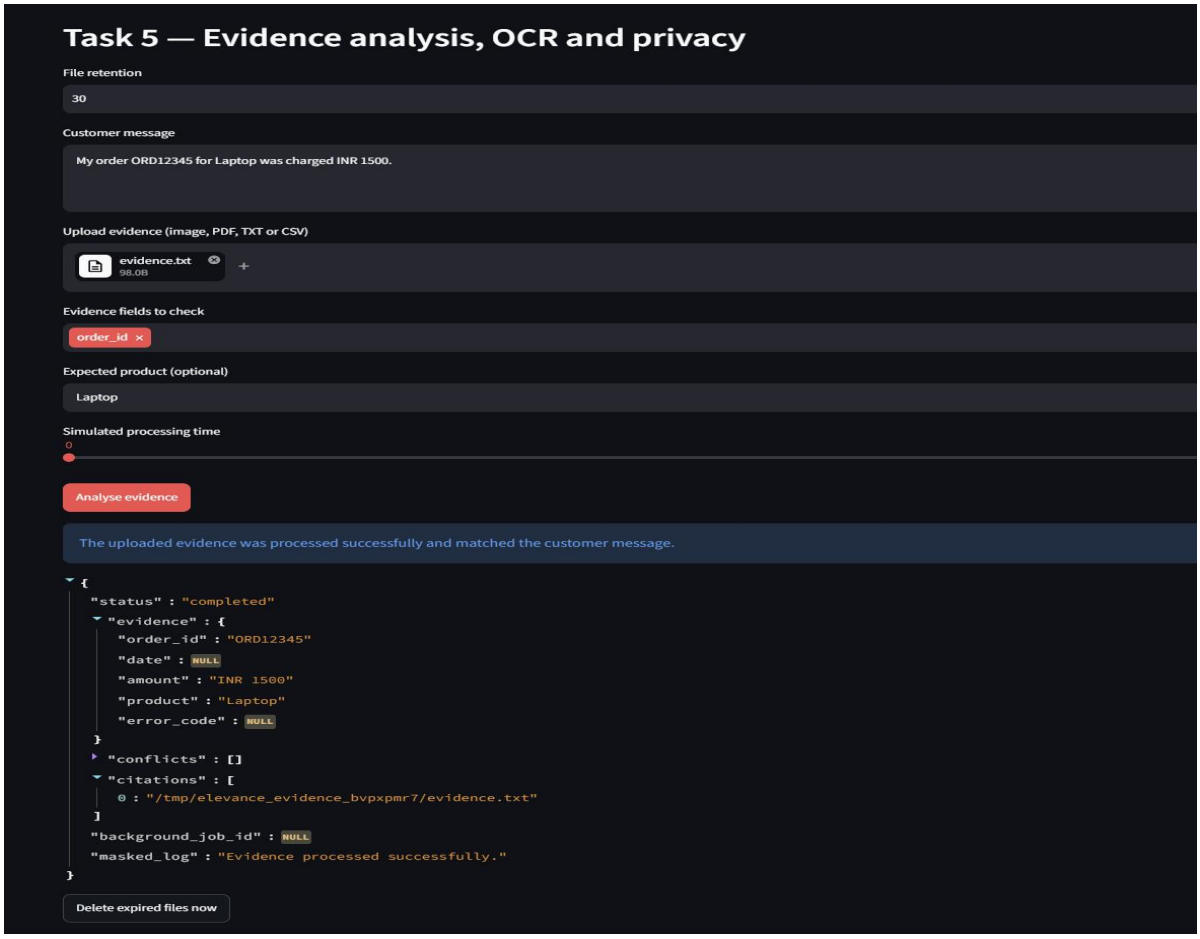


Figure 5: Task 5

7.6 TASK 6 — MULTILINGUAL CONVERSATION AND SESSION MANAGEMENT

Objective

To improve continuity across customer conversations, including language changes, and to prevent context from leaking between separate customer sessions.

Key Requirements

- Demonstrate baseline language detection for English, Hindi, Marathi, and Spanish.
- Handle language switching while preserving important entities such as order IDs.
- Maintain context across at least ten messages where configured and supported.
- Isolate customer sessions.
- Request clarification when confidence is low.
- Apply inactivity handling and session restoration rules according to configuration.

Implementation Approach

Each customer interaction should be associated with a distinct session. Relevant conversational context can be retained within that session, while language changes should not remove important facts already established. Session restoration and inactivity expiry should follow explicit rules, and sensitive context should not be shared across customers.

Result

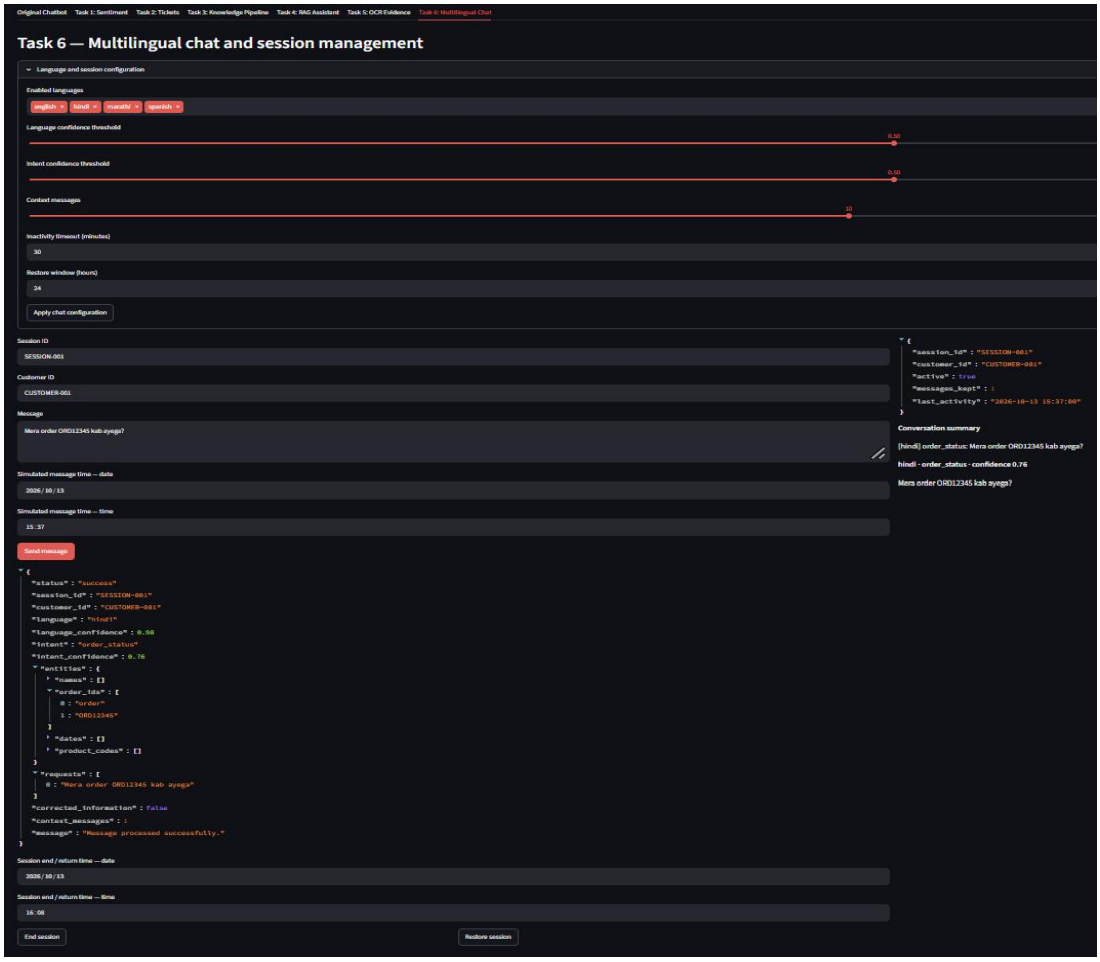


Figure 6: Task 6

8. TESTING AND VALIDATION APPROACH

Testing is important because a chatbot may produce a fluent answer even when the classification, retrieved evidence, extracted field, or routing decision is incorrect. The following test plan can be used to validate the current repository. Record actual pass/fail outcomes after running the tests in the target environment.

Test area	Representative checks	Evidence to record
Application startup	Dependencies installed; Streamlit app starts; no blocking import/configuration errors.	Terminal output and application screenshot.
Task 1	Sentiment categories, confidence handling, multilingual and urgent messages.	Input/output examples and test results.
Task 2	Required fields, priority, routing, duplicates, missing information.	Example ticket records and routing outcomes.
Task 3	New/changed/duplicate documents, invalid files, retry and rollback.	Pipeline logs or test output.
Task 4	Retrieval relevance, citations, insufficient evidence, access filtering.	Question, retrieved source, answer, and citation.
Task 5	OCR extraction, unclear fields, unsupported uploads, privacy handling.	Test files and extracted field comparisons.
Task 6	Language switching, context retention, session isolation, timeout.	Conversation transcripts and session tests.
Integration	Task modules operate with the configured interface and do not break one another.	End-to-end test notes.

8.1 Reporting Test Results

The included test runner was executed using `python tests/run_all_tests.py`. The runner returned exit code 0 and reported that all six included task test suites passed. Python syntax compilation also completed successfully.

9. CHALLENGES AND SOLUTIONS

Challenge	Practical handling / solution
Ambiguous customer messages	Use conversation context, confidence checks, and clarification questions instead of assuming a single interpretation.
Urgent or emotionally charged messages	Use clear escalation rules and test both missed escalations and false alarms.
Incomplete ticket details	Validate mandatory fields and request missing information before finalising the ticket.
Duplicate or outdated knowledge documents	Track document identity and versions; test duplicate detection and rollback paths.
Unsupported or weak retrieval results	Check retrieval relevance, filter by metadata where available, and state when evidence is insufficient.
LLM or embedding quota limits	Show clear errors, avoid marking failed processing as successful, and provide controlled retry or recovery.
OCR mistakes	Validate extracted fields, handle low-confidence values, and ask for confirmation when evidence is unclear.
Multilingual context loss	Preserve key entities and session state during language changes; test with multi-turn conversations.

Session privacy	Use session-specific context and test that one customer cannot access another session's details.
Dependency/configuration issues	Document setup steps, pin compatible dependencies where appropriate, and test in a clean environment.

10. FEEDBACK AND EVIDENCE

The internship required maintaining daily progress records and submitting the completed project along with the source code and report.

Evidence of project completion includes:

1. Internship offer letter
2. Internship task assignment email
3. Completed Python source code
4. Processed datasets
5. Streamlit dashboard
6. Individual task visualizations
7. GitHub repository
8. Deployed dashboard
9. Dashboard screenshots
10. Daily internship reports

The internship terms also mention daily reporting, original work and submission of project-related deliverables.

Evidence to include in the report

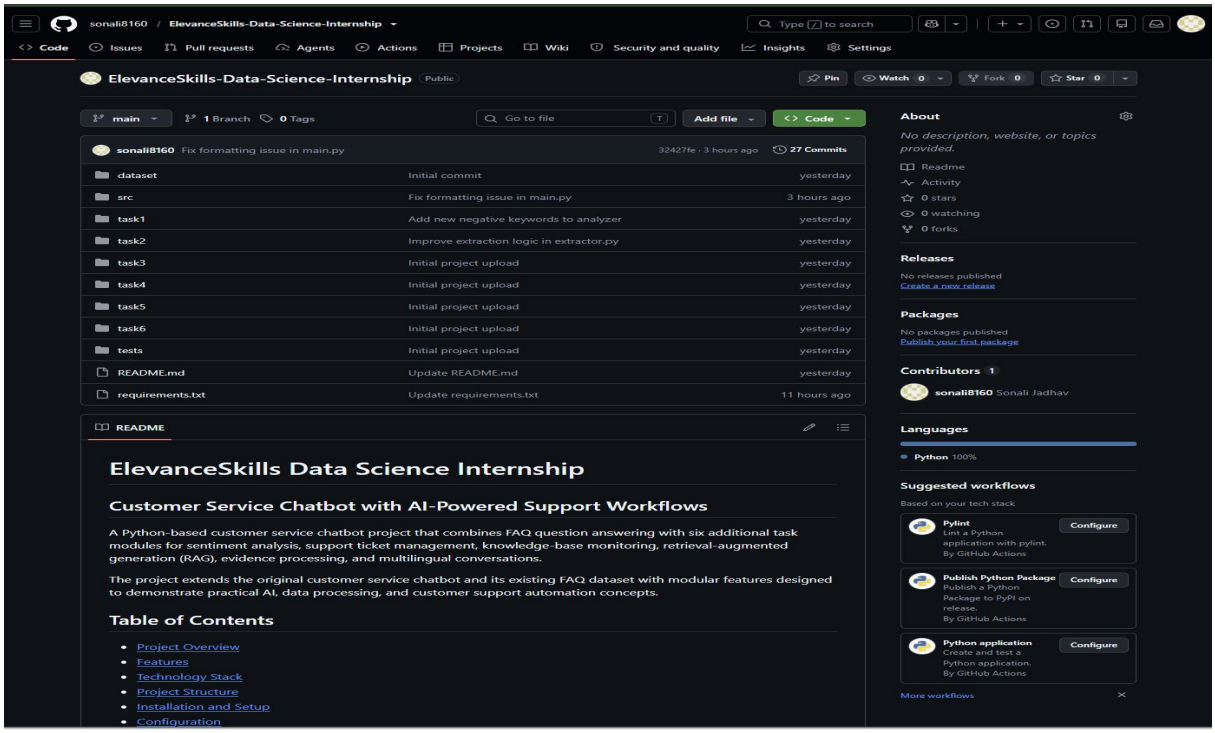


Figure 7: Project GitHub Repository

Customer Service Chatbot

Existing training project integrated with all six internship task demonstrations.

Original Chatbot | Task 1: Sentiment | Task 2: Tickets | Task 3: Knowledge Pipeline | Task 4: RAG Assistant | Task 5: OCR Evidence | Task 6: Multilingual Chat

Original Customer Service Chatbot

Use the training project's knowledge base to ask customer service questions.

Create Knowledgebase

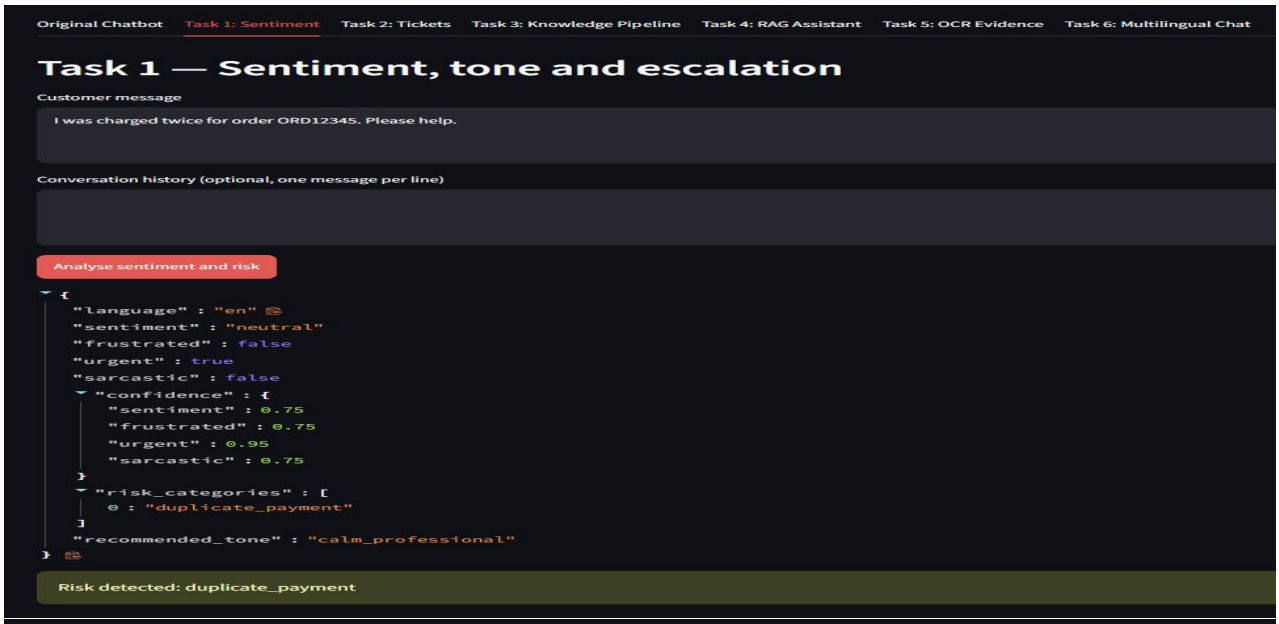
Question

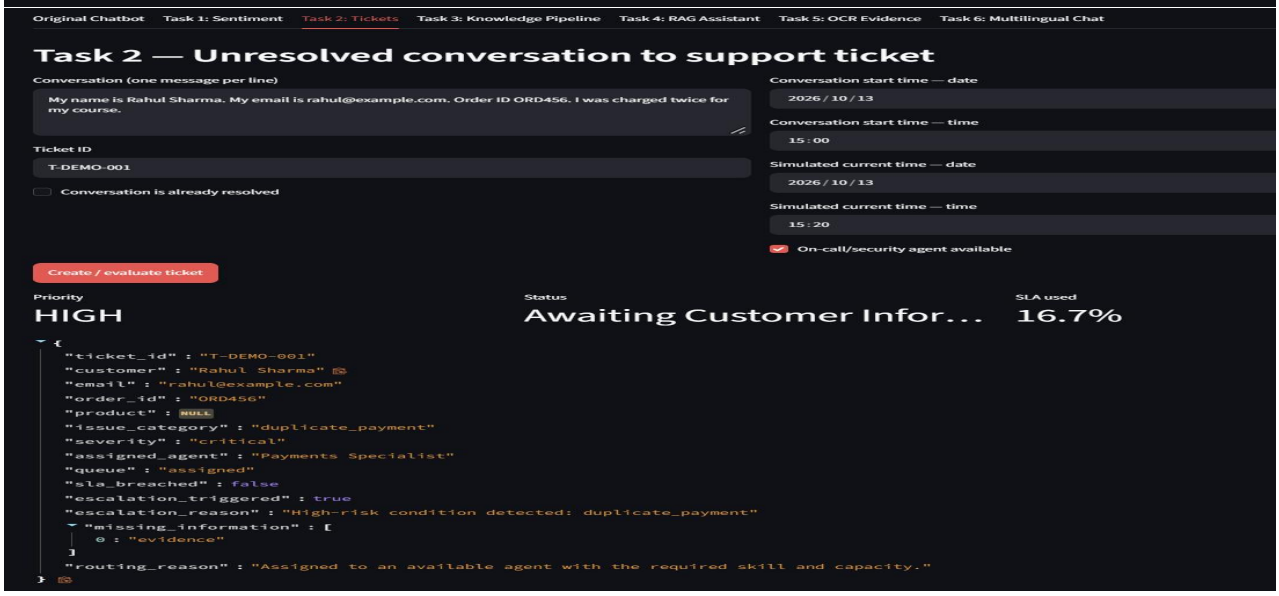
who are you

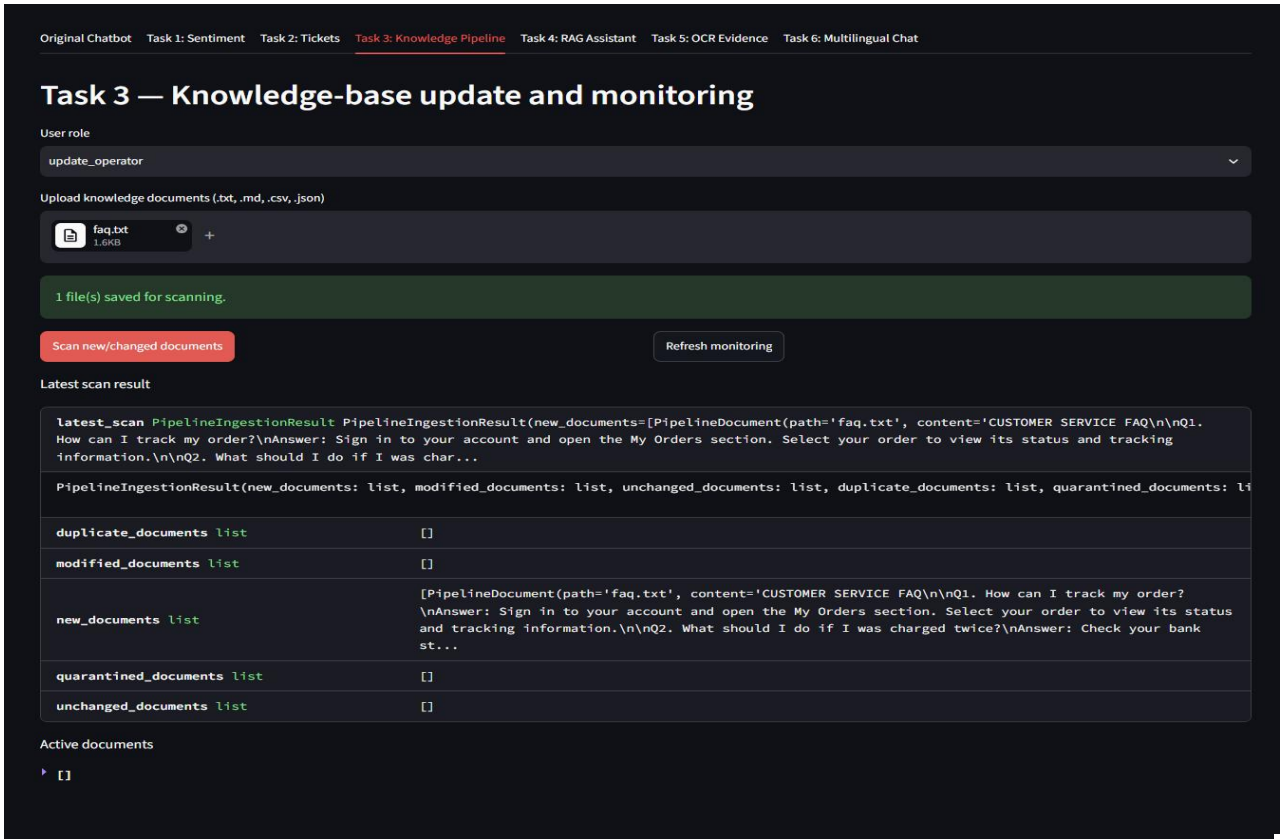
Get Answer

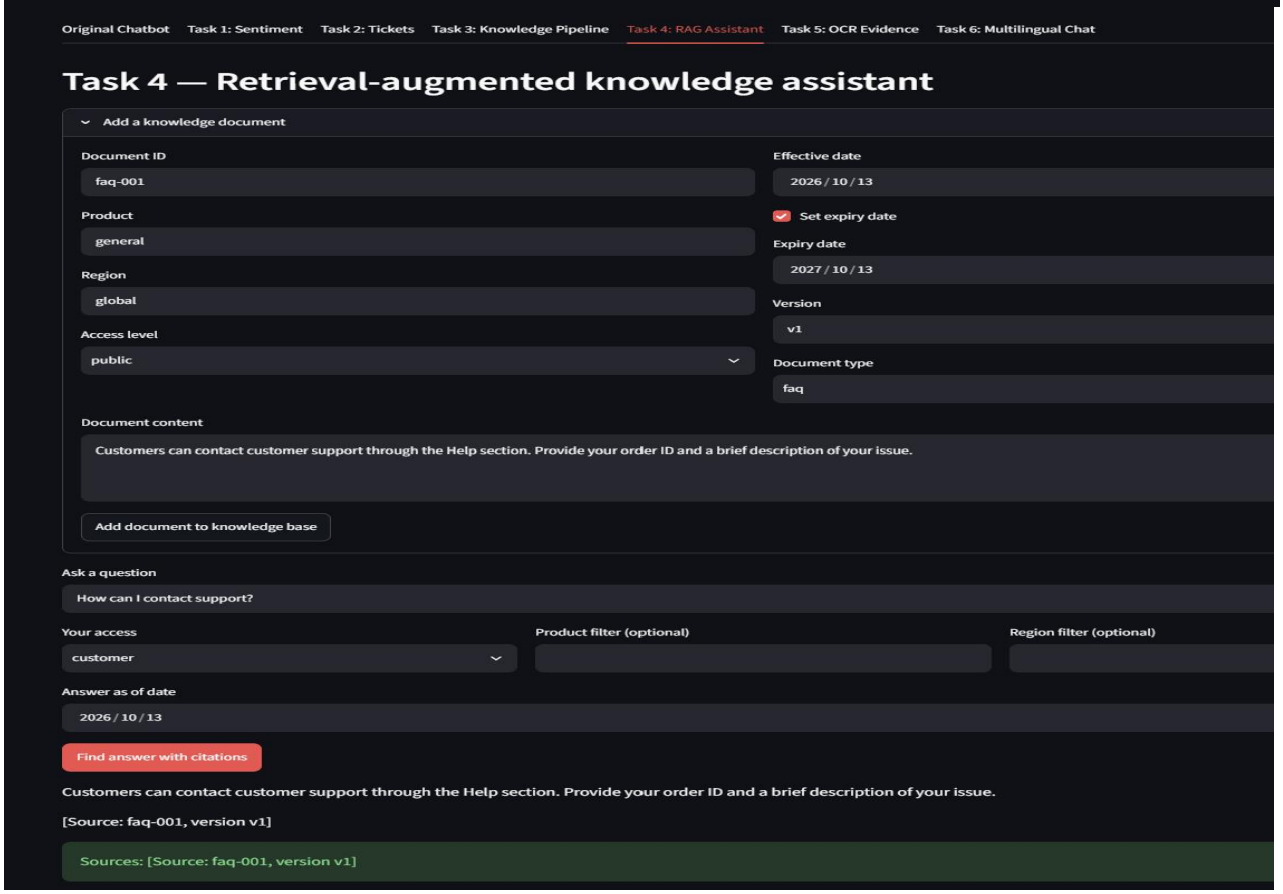
Answer

Hi there, I am your virtual assistant developed by Nullclass, I'm here to provide response for your queries related to Nullclass.






Original Chatbot
Task 1: Sentiment
Task 2: Tickets
Task 3: Knowledge Pipeline
Task 4: RAG Assistant
Task 5: OCR Evidence
Task 6: Multilingual Chat
Task 3 — Knowledge-base update and monitoring
User role
update_operator
Upload knowledge documents (.txt, .md, .csv, .json)
faq.txt
1.6KB
1 file(s) saved for scanning.
Scan new/changed documents
Refresh monitoring
Latest scan result
latest_scan PipelineIngestionResult PipelineIngestionResult(new_documents=[PipelineDocument(path='faq.txt', content='CUSTOMER SERVICE FAQ\n\nQ1. How can I track my order?\nAnswer: Sign in to your account and open the My Orders section. Select your order to view its status and tracking information.\n\nQ2. What should I do if I was char...
PipelineIngestionResult(new_documents: list, modified_documents: list, unchanged_documents: list, duplicate_documents: list, quarantined_documents: li
duplicate_documents list
[]
modified_documents list
[]
new_documents list
[PipelineDocument(path='faq.txt', content='CUSTOMER SERVICE FAQ\n\nQ1. How can I track my order? \nAnswer: Sign in to your account and open the My Orders section. Select your order to view its status and tracking information.\n\nQ2. What should I do if I was charged twice?\nAnswer: Check your bank st...
quarantined_documents list
[]
unchanged_documents list
[]
Active documents
[]


Original Chatbot
Task 1: Sentiment
Task 2: Tickets
Task 3: Knowledge Pipeline
Task 4: RAG Assistant
Task 5: OCR Evidence
Task 6: Multilingual Chat
Task 4 — Retrieval-augmented knowledge assistant
Add a knowledge document
Document ID
faq-001
Effective date
2026 / 10 / 13
Product
general
Set expiry date
Expiry date
2027 / 10 / 13
Region
global
Version
v1
Access level
public
Document type
faq
Document content
Customers can contact customer support through the Help section. Provide your order ID and a brief description of your issue.
Add document to knowledge base
Ask a question
How can I contact support?
Your access
customer
Product filter (optional)
Region filter (optional)
Answer as of date
2026 / 10 / 13
Find answer with citations
Customers can contact customer support through the Help section. Provide your order ID and a brief description of your issue.
[Source: faq-001, version v1]
Sources: [Source: faq-001, version v1]

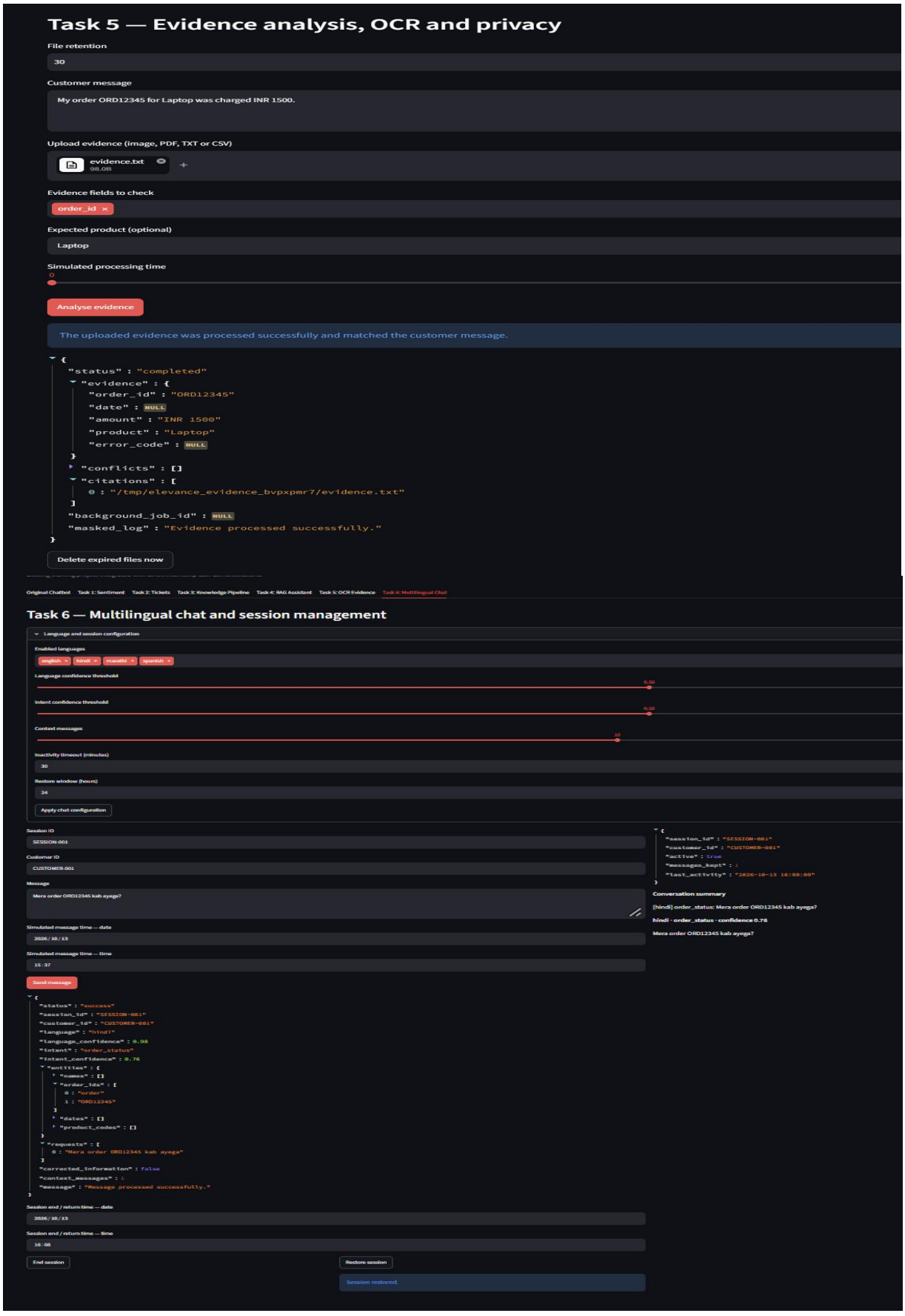


Figure 8: Deployed Dashboard

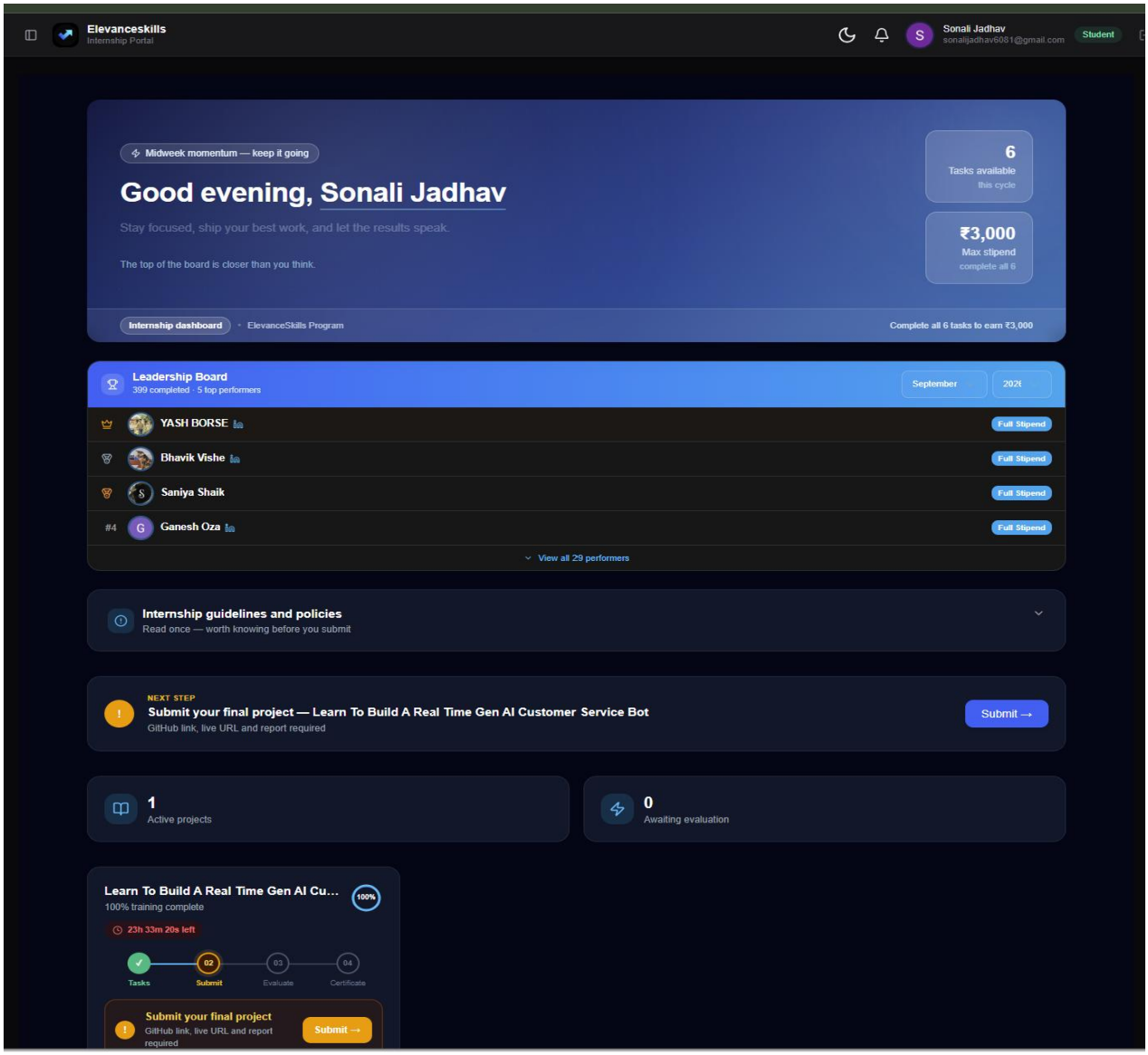


Figure 9: Internship Project Portal

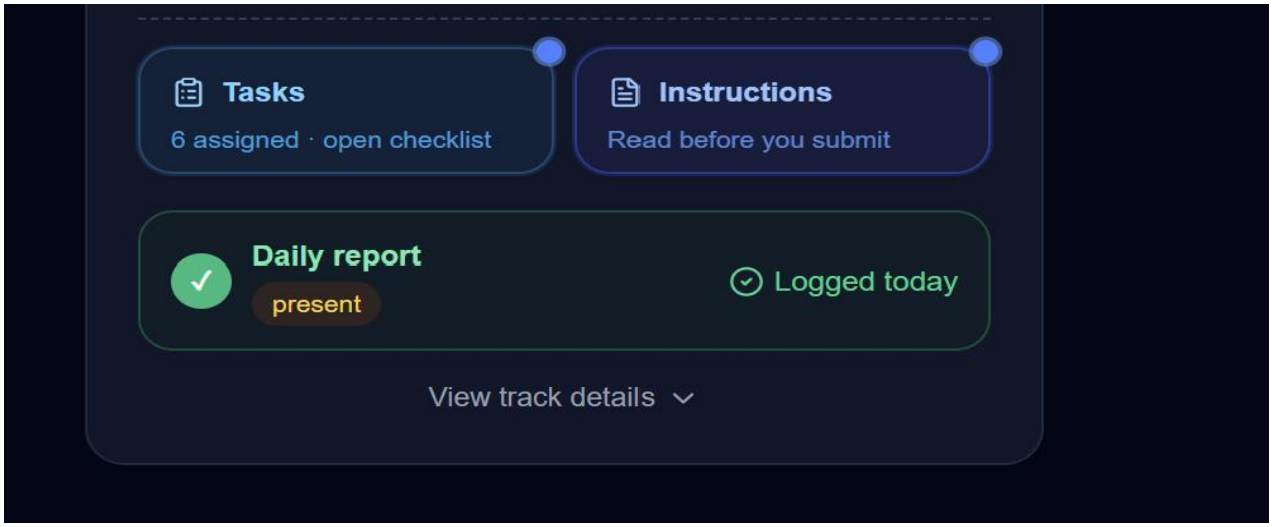


Figure 10: Internship Progress Evidence

11. OUTCOMES AND IMPACT

11.1 Project Outcomes

The project successfully integrates a customer service chatbot with six additional task modules using Python and Streamlit. It demonstrates how Generative AI, Natural Language Processing (NLP), document processing, and session management can support customer service operations.

The major outcomes of the project are:

- Intelligent Customer Support: Developed a chatbot interface to help answer customer queries using available knowledge sources.
- Sentiment Analysis: Implemented sentiment and urgency analysis to identify negative or high-priority customer messages and support appropriate escalation.
- Automated Support Tickets: Developed a workflow to extract customer and issue details, assign ticket priorities, and support agent routing and SLA monitoring.
- Knowledge-Base Management: Implemented document ingestion and monitoring to identify new, modified, duplicate, unchanged, and invalid documents.
- RAG-Based Question Answering: Developed a retrieval-augmented assistant that retrieves relevant knowledge and provides answers with source citations.
- Evidence Analysis and OCR: Implemented evidence processing to extract relevant information, compare it with customer messages, identify conflicts, and support privacy-related controls.
- Multilingual Chat and Session Management: Demonstrated language and intent detection, entity extraction, conversation context management, and session restoration.
- Integrated Dashboard: Combined the original chatbot and six task modules into a unified Streamlit application for easier testing and demonstration**.**

11.2 Project Impact

The project demonstrates the potential of AI-powered customer service systems to improve support workflows and reduce repetitive manual activities.

- Improved Response Support: Helps customer support teams find relevant information and respond to common queries more efficiently.
- Better Issue Prioritisation: Sentiment and urgency analysis can help identify complaints that require faster attention.
- Efficient Ticket Management: Structured ticket generation can reduce manual data entry and improve issue tracking.
- Reliable Knowledge Retrieval: Source citations help users verify answers against authorised documents.
- Improved Evidence Handling: Automated information extraction and comparison can support the verification of customer-provided evidence.
- Multilingual Accessibility: Support for multiple languages can make customer interactions more accessible to a wider range of users.

11.3 Overall Outcome

The project provides a practical demonstration of an integrated AI-based customer service solution. Testing confirmed successful outputs for the demonstrated chatbot workflows, including knowledge retrieval with citations, text-based evidence extraction, Hindi language and intent detection, and session restoration.

The system can be further improved through extensive testing with different customer queries, scanned documents, language variations, access permissions, and session scenarios. With additional validation and deployment, it could serve as a foundation for a more comprehensive customer support application.

12. CONCLUSION

The ElevanceSkills Data Science Internship provided an opportunity to explore a practical AI-powered customer service use case. The project combines a chatbot interface with six task areas: multilingual sentiment and risk analysis, automated ticket management, knowledge-base monitoring, retrieval-augmented generation, OCR-based evidence processing, and multilingual conversation/session management.

The work highlights that a reliable customer-support assistant requires more than generating natural-sounding responses. It also needs structured inputs, relevant and current knowledge, evidence-aware answers, clear escalation rules, careful handling of uploaded files, and session isolation. Testing and documentation are necessary to confirm that these behaviours work as intended.

Overall, the project strengthened my understanding of Python-based AI application development, modular problem-solving, retrieval workflows, document processing, testing, and GitHub-based project submission. Further improvements can focus on measured evaluation, stronger reliability controls, and deployment readiness.

13. REFERENCES

8. ElevanceSkills internship project requirements and task descriptions.
9. Project source code and README: https://github.com/sonali8160/ElevanceSkills-Data-Science-Internship
10. Python documentation: https://docs.python.org/3/
11. Streamlit documentation: https://docs.streamlit.io/
12. LangChain documentation: https://python.langchain.com/docs/
13. FAISS documentation and repository: https://github.com/facebookresearch/faiss
14. Git documentation: https://git-scm.com/doc
15. GitHub documentation: https://docs.github.com/
16. Pytest documentation: https://docs.pytest.org/

14. APPENDIX AND SUBMISSION EVIDENCE

Appendix A — Project Files

The final project consists of the following major files and folders:

- **src/internship_dashboard.py** — Main Streamlit dashboard integrating the original chatbot and all six internship tasks.
- **app.py** — Main application entry point, if included in the project.
- **src/** — Contains the source code for chatbot functionality and task implementations.
- **data/** — Stores sample datasets, knowledge-base documents, and supporting files, if included.
- **evidence.txt** — Sample evidence file used to test Task 5.
- **faq.txt** — Sample FAQ document used to test Task 3 document ingestion.
- **requirements.txt** — Lists the Python libraries and dependencies required to run the project.
- .**env** — Example configuration for environment variables, if included.
- **README.md** — Contains project information, installation instructions, execution steps, and usage guidelines.

Appendix B — Dashboard Screenshots

The following screenshots demonstrate the implementation and testing of the project:

Original Customer Service Chatbot — Demonstrates the chatbot interface and question-answering functionality.

Task 1: Multilingual Sentiment Analysis — Demonstrates sentiment classification, confidence scores, and escalation handling.

Task 2: Support Ticket Generation — Demonstrates ticket creation, priority assignment, agent routing, and SLA monitoring.

Task 3: Knowledge-Base Update and Monitoring — Demonstrates document ingestion, duplicate detection, invalid-file quarantine, and scan results.

Task 4: Retrieval-Augmented Generation (RAG) Assistant — Demonstrates knowledge retrieval, document filtering, and source citations.

Task 5: Evidence Analysis, OCR and Privacy — Demonstrates evidence processing, information extraction, matching, and privacy-related controls.

Task 6: Multilingual Chat and Session Management — Demonstrates language detection, intent classification, context management, and session restoration.

Complete Dashboard — integrated application with all six internship tasks.

Appendix C — Source Code

The complete project source code is maintained in the GitHub repository:
The repository contains the application code, task implementations, dependency information, sample files, and project documentation, as applicable. It provides a reference for reviewing the implementation and reproducing the project in a suitable Python environment.

Appendix D — Internship Evidence

The following documents and screenshots may be attached as supporting evidence of internship participation and project completion:

Include:

- Offer letter
- Task assignment email
- Internship portal screenshots
- Daily report evidence
- GitHub repository screenshot
- Deployment screenshot